Prebuilt Best Practices

When you use processes directly, small problems pop up that can lead to errors. For example, you could send a process an incorrect message that it doesn't receive. If you did this, the process mailbox would slowly fill up and exhaust your system memory. GenServer builds in best practices to ensure things like this don't happen.

Now, it's time to create a GenServer. It's actually easier than creating a process!

Write Your First GenServer

We're going to focus on four different functions in this example: init, handle_cast, handle_call, and handle_info. These are special function names that GenServer uses to call your code properly.

Create lib/examples/gen_server/simple_server.ex and add the following code:

elixir_examples/lib/examples/gen_server/simple_server.ex

```
Line 1  defmodule Examples.GenServer.SimpleServer do
        use GenServer
        require Logger

    5   def start_link(init_args, name: name) do
          GenServer.start_link(__MODULE__, init_args, name: name)
        end

        def init(speaker: speaker) do
   10     {:ok, %{speaker: speaker, last_result: nil}}
        end

        def handle_cast(
          :announce,
   15     state = %{speaker: speaker, last_result: result}
        ) do
          Logger.info("#{speaker}: The last result I computed was #{result}")
          {:noreply, state}
        end

   20
        def handle_call({:add, a, b}, _from, state) do
          result = a + b
          {:reply, result, %{state | last_result: result}}
        end
   25 end
```

Spin up an IEx instance to test this out:

```
$ iex -S mix
iex> alias Examples.GenServer.SimpleServer
iex> {:ok, pid} = SimpleServer.start_link([speaker: "Genny"], name: nil)
{:ok, #PID<0.149.0>}
```